

Parcial plantilla

```
1  public class Renta {
2      private Vehiculo vehiculo;
3      private Cliente cliente;
4      private int diasRenta;
5      private String tipoRenta;
6      private int kilometrajeInicial;
7
8      public Renta(Vehiculo vehiculo, Cliente cliente, int diasRenta) {
9          this.vehiculo = vehiculo;
10         this.cliente = cliente;
11         this.diasRenta = diasRenta;
12         this.kilometrajeInicial = vehiculo.getKilometraje();
13         this.tipoRenta = "BASICO";
14     }
15
16     public void setTipoRenta(String tipoRenta){
17         this.tipoRenta = tipoRenta; }
18
19     public double calcularTotal() {
20         if (this.tipoRenta == "BASICO") {
21             int kilometrosRecorridos = vehiculo.getKilometraje() -
this.kilometrajeInicial;
22             double precio = diasRenta * vehiculo.getPrecioPorDia()
23                 + kilometrosRecorridos *
vehiculo.getPrecioPorKm();
24
25             double adicional = 1;
26             // los autos más viejos tienen un 15% de descuento
27             if (vehiculo.getAntiguedad() > 5) {
28                 adicional = 0.85;
29             }
30             return precio * adicional;
31         }
32         else if (this.tipoRenta == "PLUS") {
33             int kilometrosRecorridos = vehiculo.getKilometraje() -
this.kilometrajeInicial;
34             return kilometrosRecorridos * vehiculo.getPrecioPorKm();
35         } else { // si el tipo de renta es "KILOMETRAJE_LIBRE":
36             return diasRenta * vehiculo.getPrecioPorDia();
37         }
38     }
39 }
```

Bad smells

- **Comments:** El código tiene la necesidad de comentarios ya que no es autoexplicativo
 - Líneas: 26 y 35
- **Long method:** El método se encarga de preguntar el tipo, calcular los kilómetros recorridos y el precio
 - Líneas 19 a 38
- **Switch statements:** Hay que preguntar por tipoRenta. a largo plazo no se expandir fácilmente
 - Líneas 20 y 32 y 25
- **Duplicate code:** Se calculan dos veces los kilómetros recorridos

Refactorings

Podemos solucionar los *Switch statements* con *Replace conditional with Strategy*, seguimos la mecánica

1. Declaro la clase TipoRenta la cual va a ser mi estrategia
2. Cambio el tipo de "tipoRenta" de String a TipoRenta
3. Hago *Move method* y paso calcularTotal a la clase "TipoRenta"
4. Implemento el método *costoTotal* en la clase TipoRenta para que la clase Renta se lo delegue a tipoRenta
5. Al método de la calcularTotal le voy a pasar los parámetros Vehículo, díasRenta y kilometrajeInicial
6. Reemplazo el tipo del parámetro del método setTipoRenta a TipoRenta
7. Aplico *Replace conditional with polymorphism*
 1. Creo las subclases Básico, Plus y KMLibre
 2. Hago que todas las subclases implementen el método costoTotal
 3. Copio el código de la rama del condicional correspondiente
 4. Elimino la parte del código del condicional
 5. Repito para cada rama del condicional
 6. Dejo el método costoTotal como abstracto
8. Después de aplicar esto nos damos cuenta que los métodos costoTotal de básico y plus quedan extensos y con variables temporales, para esto podemos hacer *extract method* de kilómetrosRecorridos, haciendo un getter, esto podemos subirlo a la superclase con un *pull up method*, para que lo usen Básico y Plus
9. Después de subir el método puedo hacer un *replace temp with query* reemplazando el método en todas las llamadas a la variable kilómetrosRecorridos

UML Final

